

Project Planning — GameStore Platform

Document version 1.0 — May 2026

1. Executive Summary

GameStore is a digital game distribution platform built on ASP.NET Core 8.0 (.NET 8) using the Model-View-Controller (MVC) architectural pattern. The platform provides a complete marketplace for purchasing, managing, and reviewing digital games, with role-based access for customers, developers, and administrators. The solution is structured across four projects: GameStore.PL (presentation layer with Razor views and controllers), GameStore.BLL (business logic layer with 20 service interfaces), GameStore.DAL (data access layer with Entity Framework Core 9.0.8 and SQL Server), and GameStore.Tests (unit tests using xUnit, Moq, and FluentAssertions, comprising 185 tests across 17 test files).

The platform implements cookie-based authentication (.GameStore.Auth cookie) with Google OAuth integration, Stripe payment processing (Checkout Sessions and webhooks), real-time features via SignalR (/hub/notifications hub handling notifications and chat), and SendGrid for transactional email delivery. Caching is handled through ASP.NET Core Output Cache (10-minute sliding expiration) with Redis fallback to in-memory distributed cache. The application is hosted on Kestrel (localhost:5000) with SQL Server and Redis provisioned via Docker Compose.

The project spans a 12-week delivery timeline from May through July 2026, organized into six phases. With 33 controllers (17 main, 11 Admin area, 5 Developer area), 62+ Razor views across three areas, and 23 database entities managed through 20 EF Core migrations, the platform delivers a comprehensive feature set including game browsing, shopping cart, wishlist, library management, developer dashboards, admin panels, chat/messaging, and a support ticket system.

2. Business Objectives

The following strategic objectives define the measurable goals for the GameStore platform launch and operation.

#	Objective
1	Deliver a production-ready ASP.NET Core MVC platform with full game catalog management, shopping cart, and checkout flow integrated with Stripe payments.
2	Implement role-based access control supporting three tiers (CUSTOMER, DEVELOPER, ADMIN) with authorization policies (AdminOnly, DeveloperOnly, CustomerOnly) and granular area-based views.
3	Achieve 185 unit tests across 17 test files covering all BLL service interfaces, using xUnit with FluentAssertions and Moq, with a minimum of 80% code coverage on business logic.
4	Maintain platform availability of 99.9% uptime with Brotli/Gzip response compression, output caching, Redis-backed distributed caching, and rate limiting on auth endpoints.
5	Provide a developer self-service portal enabling game uploads, sales management, revenue tracking, and review monitoring through dedicated Developer area views.
6	Establish real-time communication features including friend requests, direct messaging, presence tracking, and notification delivery via SignalR hub.

3. Project Timeline

The project runs over 12 weeks from May through July 2026, organized into six phases.

Phase	May 2026				Jun 2026				
	W1	W2	W3	W4	W5	W6	W7	W8	W9
Requirements Analysis									
System Design									
Implementation									
Testing									
Deployment									
Maintenance									

4. Development Phases

Each phase of the project is scoped with specific activities and expected deliverables.

Phase	Duration	Activities	Deliverables
Requirements Analysis	Weeks 1–2	Stakeholder interviews, competitor analysis, user story mapping for customer/developer/admin personas, feature prioritization	PRD document, user stories backlog, wireframe mockups for 62+ Razor views across 3 areas
System Design	Weeks 3–4	Architecture design (4-project solution), database schema with	System architecture

Phase	Duration	Activities	Deliverables
		23 entities, EF Core entity relationships, MVC route planning, SignalR hub design	diagram, ERD, route map, SignalR hub specification
Implementation	Weeks 5–8	Controller and view development across 33 controllers, EF Core migrations (20 total), Stripe integration (Checkout + Webhooks), SignalR hub, Auth + Google OAuth, SendGrid, Redis caching, rate limiting	Working application (alpha), 185 unit tests, 20 EF Core migrations, CI pipeline
Testing	Weeks 9–10	Integration testing, rate limiter validation, Stripe webhook testing, load testing with output cache and Redis, security audit of cookie auth	Test reports, performance benchmarks, bug tracker, security review
Deployment	Week 11	Production environment setup, Docker Compose deployment (SQL Server + Redis + Kestrel), HTTPS configuration, DNS setup, Stripe webhook endpoint registration	Production instance on localhost:5000, deployment runbook, Docker Compose configuration
Maintenance	Week 12 onward	Bug fixes, performance tuning, Redis cache tuning, Stripe webhook reliability monitoring, SendGrid deliverability tracking	Sprint maintenance logs, patch releases, updated documentation

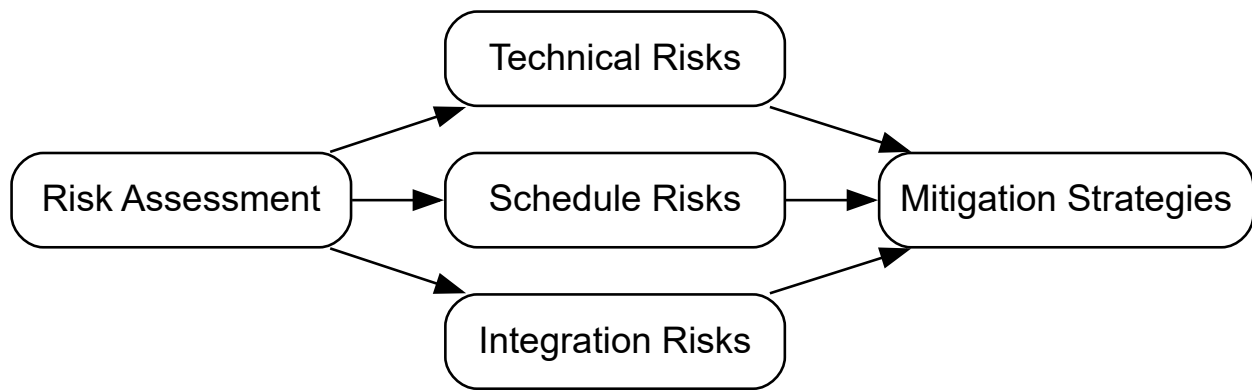
5. Task Assignment

Roles and responsibilities for the core project team.

Role	Responsibilities	Deliverables
Full-Stack Developer	Implement 33 MVC controllers and 62+ Razor views across 3 areas, manage EF Core schema with 23 entities, integrate Stripe Checkout + Webhooks, configure SignalR hub for notifications and chat	Working ASP.NET Core 8.0 application, 20 EF Core migrations, Stripe payment pipeline, SignalR hub
Backend / Database Developer	Design and build DAL (20 service interfaces, Unit of Work pattern), manage database schema and migrations, implement BLL business logic, configure Redis caching, set up rate limiting	GameStore.DAL + GameStore.BLL projects, Redis integration, rate limiter policies, 185 unit tests
DevOps Engineer	Set up Docker Compose with SQL Server and Redis containers, configure Kestrel hosting on localhost:5000, implement Brotli/Gzip compression, manage Stripe webhook endpoints	docker-compose.yml, CI/CD pipeline, deployment runbook, health check endpoint
Tester	Write and execute 185 unit tests with xUnit, Moq, and FluentAssertions; test auth flows (cookie + Google OAuth), Stripe webhook idempotency, SignalR real-time messaging, rate limiter edge cases	17 test files, test execution reports, bug tracker, QA sign-off

6. Risk Management

The diagram below maps the risk assessment categories and their relationships to mitigation strategies.



Detailed risk register identifying potential project risks along with probability, impact, and planned mitigation measures.

Risk	Probability	Impact	Mitigation Strategy
Stripe webhook delivery failure or signature validation errors	Medium	High	Implement webhook idempotency with Stripe.EventId; configure retry logic in StripeWebhookController; log all webhook payloads for manual reconciliation
SendGrid email delivery delays or bounces affecting auth flow (email verification, password reset)	Medium	High	Implement email sending retry with exponential backoff; monitor SendGrid activity API; provide manual resend option in UI with rate limit (3/15min)
EF Core migration conflicts from schema changes across 20 migrations	Low	High	Use migration history snapshots; test all migrations against staging DB before production; maintain rollback scripts for each migration
Redis cache connection failure	Low	Medium	Implement fallback to in-memory distributed cache when Redis is

Risk	Probability	Impact	Mitigation Strategy
causing performance degradation			unavailable; monitor Redis health via docker-compose healthcheck
Rate limiting blocking legitimate users under peak load	Medium	Medium	Set conservative rate limits (Login 10/min, Register 3/10min, Resend 3/15min, Verify 10/min); monitor 429 responses; whitelist known IPs for beta testing
Cookie authentication session hijacking via XSS	Low	Critical	Enforce HttpOnly, Secure, SameSite=Lax on all cookies (.GameStore.Auth, .GameStore.Session); implement anti-forgery token on all POST requests; set X-Content-Type-Options and X-Frame-Options headers
SignalR connection churn under high concurrent chat usage	Medium	Medium	Implement ConnectionTracker for connection lifecycle management; limit message payload size; configure Azure SignalR Service for scale in production
Game file upload failures (100 MB limit exceeded or disk space)	Low	High	Enforce 100 MB request body limit in Kestrel + FormOptions; validate file type server-side; implement chunked upload for larger files; monitor disk usage on gamefile volume

Risk	Probability	Impact	Mitigation Strategy
Google OAuth breaking changes or callback URL mismatches	Low	Medium	Register correct callback path (/auth/google-callback) in Google Console; implement error logging in external login flow; provide local account fallback
Brotli/Gzip compression causing response corruption on legacy proxies	Low	Low	Enable compression only for known MIME types; test compressed responses across browsers; provide Accept-Encoding negotiation

7. KPI Metrics

Key performance indicators that will be tracked post-launch to measure platform success and guide iterative improvements.

Metric Name	Target Value	Measurement Method
Registered Users	1,000 within 90 days	User entity count via database query
Daily Active Users (DAU)	300 within 90 days	Session tracking via SignalR ConnectionTracker presence data
Completed Orders	100 per day	Order entity aggregate with PaymentStatus.Completed filter
Platform Uptime	99.9% monthly	Health check endpoint (/health) monitored via external probe

Metric Name	Target Value	Measurement Method
Average Response Time	< 200 ms	ASP.NET Core logging with response time on all controller actions
Output Cache Hit Rate	≥ 60%	Cache hit/miss counters from Output Cache middleware
Stripe Checkout Conversion	≥ 55%	Stripe Dashboard funnel (session created to payment completed)
SignalR Message Delivery Latency	< 500 ms (p95)	Client-side timing on ReceiveMessage event
Email Verification Rate	≥ 70%	Users with EmailConfirmed = true / total registered users
Password Reset Success Rate	≥ 80%	PasswordResetToken usage events vs total reset requests
Developer Onboarding Rate	10 developers in first month	DeveloperApplications with ApplicationStatus.Approved
Unit Test Code Coverage	≥ 80% on BLL services	Coverlet collector reports from test runs